Latest updates: https://dl.acm.org/doi/10.1145/3705328.3748048

RESEARCH-ARTICLE

Lasso: Large Language Model-based User Simulator for Cross-Domain Recommendation

YUE CHEN, Sichuan University, Chengdu, Sichuan, China

SUSEN YANG, Kuaishou, Beijing, China

TONG ZHANG, Sichuan University, Chengdu, Sichuan, China

CHAO WANG, Kuaishou, Beijing, China

MINGYUE CHENG, University of Science and Technology of China, Hefei, Anhui, China

CHENYI LEI, Kuaishou, Beijing, China

View all

Open Access Support provided by:

University of Science and Technology of China

Sichuan University

Kuaishou

PDF Download
3705328.3748048.pdf
07 January 2026
Total Citations: 0
Total Downloads: 2366

Published: 22 September 2025

Citation in BibTeX format

RecSys '25: Nineteenth ACM Conference on Recommender Systems
September 22 - 26, 2025
Prague, Czech Republic

Conference Sponsors:
SIGCHI

RecSys '25: Proceedings of the Nineteenth ACM Conference on Recommender Systems (September 2025)
https://doi.org/10.1145/3705328.3748048
ISBN: 9798400713644

Lasso: Large Language Model-based User Simulator for Cross-Domain Recommendation

Yue Chen*†
College of Computer Science, Sichuan University
Chengdu, China
chenyueeee24@gmail.com

Susen Yang*
kuaishou Technology
Hangzhou, China
susen@mail.ustc.edu.cn

Tong Zhang
College of Computer Science, Sichuan University
Chengdu, China
scu.zhangtong@gmail.com

Chao Wang
Kuaishou Technology
Hangzhou, China
wang_chao@alumni.sjtu.edu.cn

Mingyue Cheng
State Key Laboratory of Cognitive Intelligence, University of Science and Technology of China
Hefei, China
mycheng@ustc.edu.cn

Chenyi Lei
Kuaishou Technology
Hangzhou, China
leichy@mail.ustc.edu.cn

Han Li
Kuaishou Technology
Hangzhou, China
lihan08@kuaishou.com

Abstract

Cross-Domain Recommendation (CDR) aims to mitigate the cold-start problem in target domains by leveraging user interactions from source domains. However, existing CDR methods often suffer from low data efficiency, as they require a substantial number of historical interactions from overlapping users for training, which is impractical in real-world scenarios. To address this challenge, we propose Lasso, a novel framework that leverages the large language model (LLM) as a user simulator to capture cross-domain user preferences based on the remarkable internal knowledge of the LLM. Specifically, we introduce a cross-domain training paradigm to fine-tune the LLM-based simulator, enabling it to simulate user behaviors in the target domain using historical interactions from the source domain. Furthermore, to enhance the efficiency and accuracy of Lasso, we propose two effective modules: Personalized Candidate Pool (PCP) and Confidence-Guided Inference (CGI). The PCP module employs cross-domain collaborative filtering to construct a tailored set of candidate items for simulating interactions of each cold-start user in the target domain, thereby improving the inference efficiency of the LLM. The CGI module utilizes confidence scores from the LLM to reduce noise in the simulated data, ensuring more accurate estimations. During the application phase, the simulated interactions serve as additional inputs for downstream recommendation models, effectively alleviating cold-start problems for users. Extensive experiments on public benchmark datasets and real-world industrial dataset demonstrate that Lasso achieves superior accuracy while requiring fewer historical interactions from overlapping users.

CCS Concepts

• **Information systems → Recommender systems**.

Keywords

Cross-domain Recommendation, User Simulator, Overlapping Users

ACM Reference Format:
Yue Chen, Susen Yang, Tong Zhang, Chao Wang, Mingyue Cheng, Chenyi Lei, and Han Li. 2025. Lasso: Large Language Model-based User Simulator for Cross-Domain Recommendation. In *Proceedings of the Nineteenth ACM Conference on Recommender Systems (RecSys '25), September 22–26, 2025, Prague, Czech Republic.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3705328.3748048

1 Introduction

Recommendation Systems (RS) have become ubiquitous across various domains, including e-commerce platforms like Amazon and movie platforms like Netflix. Traditional RS approaches infer user preferences from historical interactions within a specific domain, such as past purchases or clicks [35, 36]. However, these methods face a significant challenge: the cold-start problem [15], which arises from the scarcity of domain-specific historical interactions for new users, making it difficult to accurately capture nuanced user preferences and deliver precise recommendations.

The recently emerging field of cross-domain recommendation (CDR) alleviates this limitation [4, 21]. CDR involves recommending items from one domain (i.e., *target domain*) to users based on

*Both authors contributed equally to this research.
†Work done during an internship at Kuaishou Technology.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
RecSys '25, Prague, Czech Republic
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1364-4/25/09
https://doi.org/10.1145/3705328.3748048

Source domain: Book **Target domain: Movie**

Figure 1: By analyzing user preferences for science fiction and superhero books, the CDR model can infer cross-domain user preferences and recommend the movie "*Iron Man*".

their preferences and historical interactions in another domain (i.e., *source domain*). For example, as illustrated in Figure 1, by leveraging a user's history of engaging with science fiction and superhero books, CDR can infer his movie preferences and recommend "*Iron Man*". Existing approaches focus on training a mapping module that transfers user preferences from the source domain to the target domain, utilizing historical interactions of overlapping users present in both domains [4, 19]. However, these methods often suffer from low **data efficiency**, as they require a substantial number of overlapping users for effective training. In real-world scenarios, significant overlap is often impractical. For instance, on platforms like Amazon, only about 5% of users are common to both source and target domains when considering the entire user base, highlighting the limited overlap in real-world settings [15, 22, 40].

To overcome this challenge, we introduce Lasso, a novel framework that leverages LLMs As uSer SimulatOrs to estimate cross-domain user preferences. Inspired by the remarkable ability of LLMs to interpret user preferences from limited interactions [2, 9, 24], Lasso enhances data efficiency by reducing the reliance for extensive overlapping users. Specifically, Lasso employs a cross-domain training paradigm to fine-tune an LLM as a user simulator, enabling it to estimate target-domain user preferences using a diverse set of historical interactions from the source domain. The overlapping user behaviors required for the fine-tuning process are typically available but sparse. The LLM can effectively adapt to this sparsity. After fine-tuning, the LLM-based simulator generates simulated positive interactions for cold-start users in the target domain offline, leveraging their actual interactions from the source domain. These simulated interactions serve as inputs for downstream recommendation models, enhancing online performance while avoiding the latency issues associated with LLMs and increasing application flexibility. To further improve the efficiency and accuracy of the LLM-based simulator, we propose two effective modules: Personalized Candidate Pool (PCP) and Confidence-Guided Inference (CGI). The motivation behind PCP is that the LLM-based simulator will infer preferences for an extensive number of candidate items during the simulation process, which can be time-consuming. PCP addresses this by constructing a smaller, personalized candidate item pool for each user using cross-domain collaborative filtering, based on the premise that users with similar preferences in the source domain will exhibit similar preferences in the target domain. The CGI module refines the process by selecting high-quality target-domain user interactions based on the simulator's prediction confidence, reducing noise and ensuring accurate estimations of user preferences in the target domain. As such, Lasso enhances domain transferability and simulates high-quality behaviors of cold-start users for conventional recommendation models.

We experimentally demonstrate the effectiveness of Lasso across various cross-domain scenarios, including widely used benchmark datasets[1] and real-world industrial dataset. Results indicate that Lasso consistently achieves superior performance improvements while relying on fewer historical interactions from overlapping users. These advantages are evident across different downstream recommendation models. We summarize our contributions as follows:

- We highlight the data efficiency challenge associated with overlapping users in the cross-domain recommendation task and, for the first time, propose the use of LLMs to tackle this challenge.
- We introduce Lasso, an effective framework that adapts the LLM as a user simulator to estimate cross-domain user preferences, enhanced by the PCP and CGI modules to improve efficiency and accuracy.
- We conduct extensive experiments on public benchmark datasets and real-world industrial dataset to validate the effectiveness of Lasso, providing insights into its performance.

2 RELATED WORK

We propose leveraging LLMs to address the data efficiency challenge in cross-domain recommendation tasks. Below, we review relevant literature and highlight the distinctions between our approach and existing methods.

2.1 Cross-Domain Recommendation

As a foundational study addressing the cold-start problem [33], existing CDR methods can be broadly categorized into two groups [3, 37]: direct mapping methods [4, 16, 21, 26, 29, 38] and meta-based mapping methods[8, 39, 40]. Direct mapping methods focus on transferring user preferences between source and target domains, heavily relying on overlapping users for training. For instance, EMCDR [21] employs matrix factorization to learn user preferences in the source domain and uses a neural network as a mapping function to transfer these preferences to the target domain. However, training this mapping function requires a significant number of overlapping users. SSCDR [15] improves upon EMCDR by incorporating deep metric learning and semi-supervised training, reducing the reliance on extensive training data. Besides, DisenCDR[5] and UniCDR [4] disentangle domain-shared and domain-specific information using mutual-information-based regularizers and contrastive loss, respectively. AMT-CDR[20] constructs a cross-domain knowledge graph and employs adversarial learning to maintain consistency across data channels, preventing negative transfer. Meta-based mapping methods, on the other hand, leverage meta-learning to enhance data efficiency when training data is scarce. For example, TMCDR [39] uses the Model-Agnostic Meta-Learning (MAML) framework [8] to optimize a meta-network as the mapping function. PTUPCDR [40] extends TMCDR by incorporating personalized mapping to model individual user preferences. While meta-learning

[1] https://jmcauley.ucsd.edu/data/amazon/

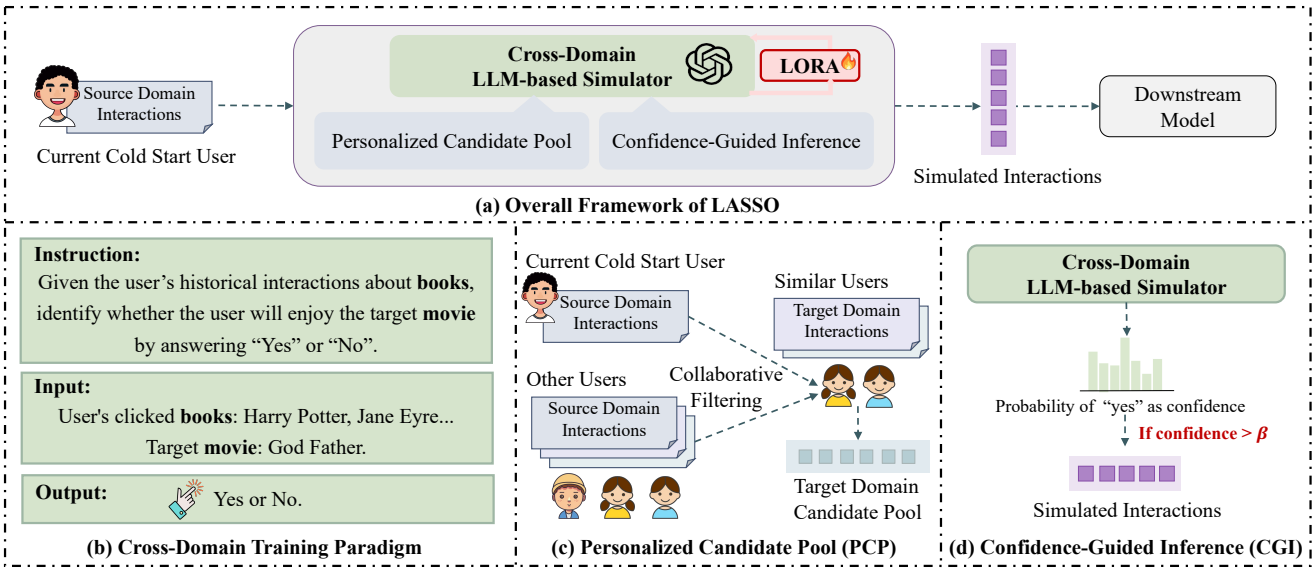

Figure 2: (a) the overall framework of Lasso, which utilizes the LLM-based simulator to generate simulated interactions in the target domain for cold-start users, enhanced by the Personalized Candidate Pool (PCP) and Confidence-Guided Inference (CGI) modules, thereby improving the performance of downstream models. (b) the cross-domain training paradigm of the LLM-based user simulator. (c) the PCP module, which employs cross domain collaborative filtering to build the personalized candidate item pool in the targe domain. (d) the CGI module, which leverages confidence scores to identify high-quality simulated interactions.

methods can reduce dependence on training data to some extent, they still require a substantial number of overlapping users [39, 40], which is often impractical in real-world scenarios. To address this limitation, we explore the potential of LLMs in CDR tasks for the first time, inspired by their remarkable ability in data efficiency [7].

2.2 LLM-based Recommendation

The rise of LLMs has spurred significant interest in their integration with recommender systems [1, 18, 34]. Early research primarily focuses on adapting LLMs as recommendation models [9, 30, 31]. achieving notable performance through in-context learning [9] and instruction tuning [30, 31]. For instance, TallRec [2] proposes a tuning framework to align LLMs with recommendations. LLM-Rec [24] mixes behaviors across different domains to explore the ability of LLMs in multi-domain behaviors modeling. The paper [23] further proposes a paradigm to fine-tune large language models for cross-domain recommendation tasks, demonstrating that LLMs can effectively bridge domain gaps by learning transferable user preferences. However, these approaches face practical challenges, particularly in terms of inference latency [17], and lack the exploration of the sparsity of cross-domain behaviors, which is common in real-world application scenarios. Recent methods have shifted towards using LLMs to support lightweight online recommendation models in an offline manner [14, 17, 28]. Notably, LLM-InS [14] employs LLMs as user simulators to address item cold-start problem. By generating simulated interactions for cold-start items, LLM-InS enhances the performance of downstream recommendation models while avoiding online inference issue. However, LLM-InS primarily relies on in-domain user behaviors, limiting its applicability to cold-start users. Additionally, LLM-InS focuses on item content information for candidate set creation, overlooking the critical role of collaborative filtering signals. In this paper, we propose exploiting cross-domain user behaviors and underscores the potential of LLM-based simulators to address user cold-start challenges, particularly in the scenario where cross-domain behaviors are sparse.

3 PRELIMINARIES

Notations. We denote the set of users as $\mathcal{U} = \{u_1, u_2, \ldots\}$, the set of items by $\mathcal{V} = \{v_1, v_2, \ldots\}$, and the observed user-item interaction pairs by $\mathcal{R} = \{(u_1, v_1), (u_2, v_2), \ldots\}$, where (u_i, v_i) indicates that user u_i likes the item v_i. We use the superscripts s and t to denote the source domain and target domain, respectively. Specifically, $\mathcal{U}^s$, $\mathcal{V}^s$, and $\mathcal{R}^s$ represent the user set, item set, and interaction pairs in the source domain, while $\mathcal{U}^t$, $\mathcal{V}^t$ and $\mathcal{R}^t$ denote these in the target domain. Note that the item sets $\mathcal{V}^s$ and $\mathcal{V}^t$ belong to distinct domains and do not overlap. We define overlapping users who have interactions in both domains as $\mathcal{U}^o = \mathcal{U}^s \cap \mathcal{U}^t$. Meanwhile, cold-start users in the target domain are denoted as $\mathcal{U}^d = \mathcal{U}^s / \mathcal{U}^t$.

Problem Definition. The essence of cross-domain recommendation is to identify the items v_m^t that a cold-start user u_i may prefer in the target domain, leveraging interaction data from the source domain, denoted as $\mathcal{R}_i^s = \{(u_i, v_1^s), (u_i, v_2^s), \ldots\}$. To achieve this, we first train an LLM as a user simulator using a cross-domain training paradigm, utilizing interaction data $\mathcal{R}^s$ and $\mathcal{R}^t$ from overlapping users $\mathcal{U}^o$ in both domains. This enables the LLM-based simulator to predict user preferences for items in the target domain based on source-domain interactions. Next, for a cold-start user $u_i \in \mathcal{U}^d$ in

the target domain, the LLM-based simulator generates simulated interaction data, represented as $\mathcal{R}_i^{t*} = \{(u_i, v_1^1), (u_i, v_2^t), \ldots\}$. We denoted the simulated item sequence as $\mathcal{V}_{u_i}^{t*} = \left\{v_k^t | (u_i, v_k^t) \in \mathcal{R}_i^{t*}\right\}$. This sequence is then used as input for downstream recommendation models, effectively mitigating data scarcity issue for cold-start users.

4 METHODOLOGY

The Lasso framework is illustrated in Figure 2(a). The framework consists of three key steps. First, we use the cross-domain training paradigm to fine-tune the LLM as a user simulator, leveraging LoRA technology [13]. Next, we construct a personalized candidate pool for each cold-start user in the target domain through cross-domain collaborative filtering. The LLM-based simulator then infers the preferences of the cold-start user for each item in the candidate pool, generating simulated interactions in the target domain using a confidence-guided inference module. Finally, these simulated interactions are fed into a downstream recommendation model to enhance online performance. In the following sections, we provide a detailed explanation of each component.

4.1 Cross-Domain Training Paradigm

Leveraging the impressive in-context learning capabilities and data efficiency potential of LLMs, we propose the development of a user simulator. To achieve this, we introduce a cross-domain training paradigm to fine-tune the LLM, enabling it to predict user preferences in the target domain based on their interaction behaviors in the source domain.

Following [3, 37], we utilize the historical interactions of overlapping users across the source and target domains as our training dataset. The interactions in the source domain serve as input, while their interactions in the target domain serve as output. To facilitate the training of LLM, we reformulate these interactions into an instruction-following format. Specifically, as illustrated in Figure 2(b), each data point consists of three components: Instruction, Input, and Output. The "Instruction" provides guidance on predicting user behavior toward a target domain item based on their source domain interactions. The "Input" includes the user's source domain interactions paired with a candidate item from the target domain, where each item is characterized by its title and key attributes (e.g., a book is defined by its title and author). The "Output" reflects the user's actual behavior toward the target item, serving as a supervisory signal. In this study, the "Output" is represented as either "yes" or "no", corresponding to "like" or "dislike" for the candidate item, respectively.

During training, we maximize the probability of "Output" by applying an auto-regressive loss and utilizing LoRA for parameter updating. Formally:

$$\mathcal{L} = -\frac{1}{|\mathcal{Z}|} \sum_{(x,y)\in\mathcal{Z}} log\left(P\left(y_t | x, y_{<t}; \Theta_h, \Theta_l\right)\right), \tag{1}$$

where x denotes the concatenation of "Instruction" and "Input", y denotes the "Output", y_t is the t-th token of y being processed by the LLM, $y_{<t}$ denotes the tokens of y already processed by the LLM, and $\mathcal{Z}$ denotes the training dataset. The "Output" y is binary, taking values of either "yes" or "no". Θ_h and Θ_l represent the fixed original parameters and the updated LoRA parameters of the LLM, respectively. After training, the fine-tuned LLM-based simulator is employed to estimate user-item interactions in the target domain.

Algorithm 1 Pseudo code of the Lasso workflow

Input: Overlapping interactions $\mathcal{R}^o$ of $\mathcal{U}^o$, source-domain interactions $\mathcal{R}_i^s$ of the cold-start user u_i, target-domain item v_m^t, threshold β, number K;

Output: Estimated preference $y_{u_i}^m$ of u_i toward item v_m^t;

1: Fine-tune the LLM as a user simulator using $\mathcal{U}^o$ as *section 4.1*;
2: Construct personalized candidate pool $\mathcal{V}_{u_i}^{t'}$ as *section 4.2*;
3: Input $\mathcal{R}_i^s$ and $\mathcal{V}_{u_i}^{t'}$ into the user simulator with β;
4: Obtain K simulated interactions $\mathcal{V}_{u_i}^{t*}$ using CGI as *section 4.3*;
5: Input u_i, v_m^t, and $\mathcal{V}_{u_i}^{t*}$ into the downstream model;
6: **return** $y_{u_i}^m$

4.2 Personalized Candidate Pool (PCP)

Evaluating whether each cold-start user will engage with every candidate item in the target domain is computationally expensive due to the large number of candidate items. To address this, we propose constructing a refined candidate pool $\mathcal{V}_{u_i}^{t'}$ for each cold-start user u_i before the LLM-based simulator generates simulated interactions. Inspired by prior work [19], which suggests that users with similar preferences in the source domain are likely to exhibit analogous preferences in the target domain, we employ a cross-domain user-based collaborative filtering approach to build the candidate pool. As illustrated in Figure 2(c), cold-start users in the target domain have historical interactions in the source domain. We begin by calculating user similarity based on their behaviors in the source domain using collaborative filtering principles. The similarity is defined as:

$$sim\left(u_i, u_j\right) = \sum_{v\in\mathcal{V}_{u_i\cap u_j}^s} \frac{1}{log\left(1+n_v\right)\cdot\sqrt{J_{u_i}\cdot J_{u_j}}}, \tag{2}$$

where $\mathcal{V}_{u_i\cap u_j}^s$ denotes the set of source-domain items liked by both u_i and u_j, n_v is the frequency of item v appearing in all interactions, J_{u_i} and J_{u_j} denote the number of items liked by users u_i and u_j, respectively. In short, the higher the overlap of items liked by users u_i and u_j in the source domain, the higher the similarity between u_i and u_j. Then, we select the top M users most similar to u_i, denoted as $\mathcal{U}^{u_i}$, and construct the candidate pool $\mathcal{V}_{u_i}^{t'}$ by aggregating items liked by these similar users in the target domain:

$$\mathcal{V}_{u_i}^{t'} = \left\{v_k^t | v_k^t \in \mathcal{V}_{u_j}^t \ \&\ u_j \in \mathcal{U}^{u_i}\right\}, \tag{3}$$

where u_j is a user similar to u_i, and $\mathcal{V}_{u_j}^t$ denotes the items liked by u_j in the target domain. The proposed selection mechanism effectively leverages source-domain behavior information to mitigate the sparsity of cold-start user interactions in the target domain. Once the candidate pool $\mathcal{V}_{u_i}^{t'}$ is constructed, it serves as input to the LLM-based simulator, which generates a simulated sequence of items likely to be preferred by the user.

4.3 Confidence-Guided Inference (CGI)

Recognizing that simulated interactions generated by the LLM may contain noise, which can negatively impact downstream model performance, we propose the Confidence-Guided Inference (CGI) mechanism. This mechanism leverages the confidence scores of the LLM to mitigate the effects of noisy data, drawing on insights from prior research [25]. These confidence scores are derived from the token probability distribution during the generation process. Additionally, experiments in Section 5.4 demonstrate a correlation between higher confidence levels in the LLM and reduced hallucination rates.

Specially, for a cold-start user u_i, we input the source-domain interactions $\mathcal{R}_i^s$ along with a target-domain candidate item $v_k \in \mathcal{V}_{u_i}^{t'}$ into the LLM-based simulator, following the input construction described in Section 4.1. The LLM estimates the user's preference for item v_k by generating an output of either "yes" or "no". We normalize the probabilities associated with the LLM's outputs, ensuring their sum equals 1. If the normalized probability of generating "yes" exceeds 0.5, the item is considered to align with the user's preferences. We denote this probability as $P_{v_k}^{u_i}$. Rather than simply selecting items with $P_{v_k}^{u_i} > 0.5$ as simulated interactions, we treat $P_{v_k}^{u_i}$ as the **confidence score** of the LLM-based simulator, following previous studies [27]. An item v_k is only included as a simulated interaction for user u_i if its confidence score $P_{v_k}^{u_i}$ exceeds a predefined threshold β. This process is formulated as:

$$\mathcal{V}_{u_i}^{t*} = \left\{ v_k^t | v_k^t \in \mathcal{V}_{u_i}^{t'} \ \& \ P_{v_k}^{u_i} > \beta \right\}, \tag{4}$$

where β is the confidence threshold, $\mathcal{V}_{u_i}^{t'}$ represents the candidate items in the target domain, and $\mathcal{V}_{u_i}^{t*}$ denotes the final set of K simulated interactions with the top confidence scores. When $\beta = 0.5$, all items for which the LLM outputs "yes" are treated as simulated interactions. Conversely, when $\beta = 1$, no items are selected, and the downstream model operates without simulated interactions.

Ultimately, The simulated interactions $\mathcal{V}_{u_i}^{t*}$ are incorporated into the downstream recommendation model as historical behaviors of cold-start users in the target domain. During online inference, when the recommendation model $\mathcal{F}$ evaluates whether user u_i is likely to prefer a specific item v_m^t in the target domain, the process is formulated as:

$$y_{u_i}^m = \mathcal{F}(u_i, v_m^t; \mathcal{V}_{u_i}^{t*}), \tag{5}$$

where $y_{u_i}^m$ represents the preference score for item v_m^t. The workflow of the Lasso is presented in Algorithm 1.

5 EXPERIMENTS

To evaluate the effectiveness of Lasso, we conduct extensive experiments on public datasets to address the following research questions:

- **RQ1:** Does Lasso achieve superior performance with fewer training data compared to existing CDR methods?
- **RQ2:** How do the individual components of Lasso contribute to its overall performance?
- **RQ3:** How do various hyper-parameters, such as the number of simulated interactions, influence the model's performance?
- **RQ4:** Is the LLM-based simulator capable of generating high-quality simulated interactions?

Table 1: Statistic of the Amazon benchmark. Only 5% of users are overlapped in the entire user base.

Tasks	Domain		Item		User		
	Source	Target	Source	Target	Source	Target	Overlap
Task1	Book	Movie	367,982	50,052	603,668	123,960	37,388
Task2	Book	Music	367,982	64,443	603,668	75,258	16,738
Task3	Movie	Music	50,052	64,443	123,960	75,258	18,031

5.1 Experimental Settings

5.1.1 Datasets. The experiments are conducted using the Amazon review dataset[2], a widely adopted benchmark for cross-domain recommendation (CDR) tasks. Following previous studies [32, 39, 40], we evaluate our approach on three cross-domain tasks: Task 1: Book → Movie, Task 2: Book → Music, and Task 3: Movie → Music. For example, in Task 1, the Book domain serves as the source domain, while the Movie domain represents the cold-start target domain. Each sample in the Amazon dataset consists of a user-item review interaction, including user information, item details, and the user's rating for the item. Following standard practice [11], we define positive interactions as those with review ratings above 3, while ratings of 3 or below are categorized as negative. To ensure consistency, we retain overlapping users for both training and evaluation. We use 60% of user-item interactions recorded before January 1, 2018 as the training set for the LLM-based simulator, downstream models, and baseline methods. The remaining 20% of interactions occurring after this date are reserved for evaluation. Notably, we filter the test set to include only cold-start users who are not part of the training set [40]. Detailed statistics of the datasets are provided in Table 1.

5.1.2 Metrics. To evaluate the performance of our approach, we adopt two widely used metrics following prior work [4, 39]: AUC and NDCG. AUC measures the overall prediction accuracy and ranking quality, while NDCG evaluates the quality of ranked item lists by considering the positions of items, emphasizing the importance of ranking order. For NDCG, we report the performance based on the top 50 ranking results.

5.1.3 Baselines. We consider the following two categories of state-of-the-art CDR methods as baselines:

- **Direct mapping methods** transfer user representations through a neural mapping function. The baselines in this category include: 1) TGT[12] employs a neural collaborative filtering module as the mapping function, trained exclusively on the target domain. 2) EMCDR[21] uses a multi-layer perceptron to capture domain-specific representations and learns a cross-domain mapping function to identify cross-domain preferences. 3) UniCDR[4] leverages both domain-specific and shared features through a unified mapping function. 4) AMT-CDR[20] constructs a cross-domain knowledge

[2] http://jmcauley.ucsd.edu/data/amazon/

Table 2: Performance of various cross-domain recommendation methods on benchmark dataset. The best results are bold and the second-best results are underlined.

Tasks & Domains	Metrics	UniCDR	TGT	EMCDR	TMCDR	PTUCDR	AMT-CDR	LASSO
Task1: Book → Movie	AUC	0.6239	0.6957	0.6581	0.6841	0.6558	0.6865	**0.7108**
	NDCG@50	0.6415	0.6755	0.7341	0.7712	0.7370	0.7623	**0.8214**
Task2: Book → Music	AUC	0.5734	0.6365	0.6026	0.6631	0.5991	0.6578	**0.6656**
	NDCG@50	0.5943	0.6330	0.5722	0.6713	0.5899	0.6636	**0.7961**
Task3: Movie → Music	AUC	0.5874	0.6633	0.6163	0.6668	0.6125	0.6669	**0.6902**
	NDCG@50	0.6105	0.6885	0.6492	0.6761	0.6060	0.6798	**0.8804**

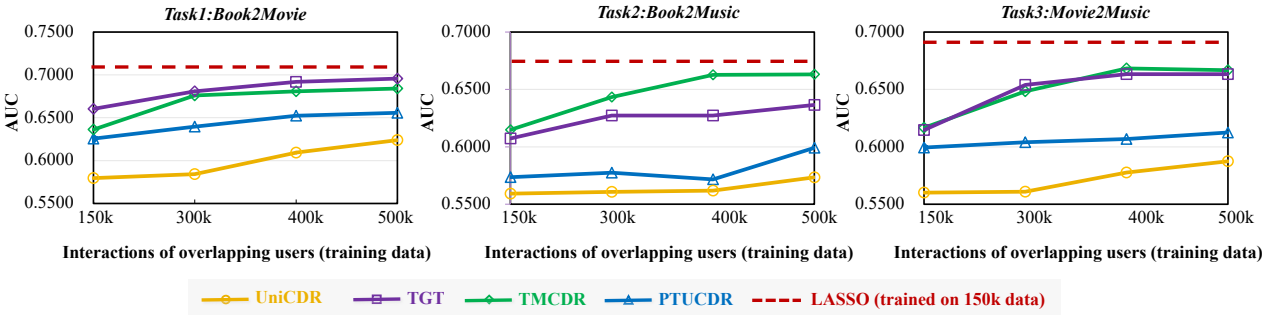

Figure 3: Performance of various recommendation methods with different training data sizes.

graph and employs adversarial learning to maintain consistency across data channels.

- **Meta-based mapping methods** integrate a meta-network to replace traditional mapping functions, thereby enhancing data efficiency. The baselines in this category include: 1) TMCDR[39] employs the Model-Agnostic Meta-Learning (MAML) framework to optimize a meta-network based on preference ratings. 2) PTUCDR[40] incorporates user-specific embeddings into a meta-network to facilitate the transfer of user preferences from the source to the target domain.

5.2 Implementation Details

The selection of specific LLMs for the user simulator is not the primary focus of this study. Given the notable success of LLAMA-7B in prior research[2, 14, 31], we adopt LLAMA-7B as the backbone for the user simulator. Additionally, we incorporate the Deep Interest Network (DIN) [36] as the downstream recommendation model, following previous studies [17]. DIN is chosen for its widespread adoption in industrial settings and its demonstrated effectiveness in modeling dynamic user behaviors. An analysis of alternative downstream models is provided in Section 5.6. The confidence threshold β and the number of interactions K are selected based on their performance on the validation set, and are subsequently used during the test phase. For training the LLM-based simulator, we sample 150,000 training instances and apply LoRA fine-tuning for 5 epochs using two A800 GPUs. The training employs the AdamW optimizer with the learning rate of 10^{-4}, the batch size of 80 and maximum sequence length of 512. For the downstream recommendation model, we use the Adam optimizer and train for 3 epochs with the dropout ratio of 0.2 and hidden layer sizes of [256, 128, 64, 1]. For baseline models, we sample 500,000 training instances and train for 10 epochs. All hyperparameters for the baseline methods are consistent with those reported in their original publications.

5.3 Overall Performance (RQ1)

We evaluate the overall performance of all methods, focusing on task effectiveness and data efficiency. The results are presented in Table 2 and Figure 3, from which we draw the following observations. As shown in Table 2, LASSO outperforms all baselines across three tasks, achieving a notable performance margin while demonstrating higher data efficiency. On average, it surpasses the leading CDR baseline, TMCDR, by approximately 2% in AUC across all tasks. To further analyze data efficiency, we evaluate the performance of each method with varying amounts of training data. As illustrated in Figure 3, LASSO consistently achieves superior performance while requiring fewer overlapping users for training. Notably, when using the same training dataset (i.e., 150K data points), LASSO outperforms the baselines. While increasing the training data from 150K to 500K improves the performance of the baselines, they still fall short of LASSO that achieves better results with only 150K data points. This observation supports our hypothesis that existing methods heavily rely on a large number of overlapping user interactions for training, whereas LASSO achieves better outcomes with fewer interactions. These results highlight LASSO's higher data efficiency for cross-domain recommendations, enhancing its practical utility.

Table 3: Recommendation performance achieved by Lasso variants.

Model	*Task1*		*Task2*		*Task3*	
	AUC	NDCG	AUC	NDCG	AUC	NDCG
Lasso$_{/S}$	0.7004	0.8156	0.6538	0.7847	0.6870	0.8655
Lasso$_{/P}$	0.7049	0.8111	0.6625	0.7958	0.6845	0.8678
Lasso$_{/C}$	0.7096	0.8147	0.6655	0.7912	0.6897	0.8724
Lasso	**0.7108**	**0.8214**	**0.6656**	**0.7961**	**0.6902**	**0.8804**

5.4 Ablation Study (RQ2)

To investigate the effectiveness of each component in Lasso, we conduct ablation studies by evaluating the performance of the following variants:

- Lasso$_{/S}$ removes the LLM-based simulator and treats all candidate items constructed by the PCP module as simulated interactions.
- Lasso$_{/P}$ replace the personalized candidate pool with randomly selected items.
- Lasso$_{/C}$ removes the CGI mechanism and treats all items for which the simulator outputs "yes" as simulated interactions.

The performance of Lasso variants across different tasks is summarized in Table 3. We have following findings. Lasso consistently outperforms Lasso$_{/S}$ on all tasks, demonstrating that the LLM-based simulator generates high-quality simulated interactions by leveraging the remarkable knowledge of the LLM, thereby enhancing the performance of downstream models. Moreover, omitting the personalized candidate pool (Lasso$_{/P}$) leads to a performance decrease across all tasks, indicating that constructing personalized candidate sets not only alleviates the time-consuming problem, but also improves simulation effectiveness by capturing cross-domain preferences. Lasso also achieves better performance than Lasso$_{/C}$, showing that confidence scores can effectively filter noise and improve the quality of simulated interactions.

Additionally, we explore the relationship between the CGI module and the reliability of knowledge transfer, demonstrating how CGI contributes to the success of Lasso. To achieve this, we evaluate the correlation coefficient between the confidence scores obtained from CGI and manually assessed reliability scores for knowledge transfer. The reliability scores are measured by requiring Lasso to output its decision rationales for each predicted user preference, which are then evaluated by human experts. Specifically, when generating simulated interactions for a cold start user u_i, we prompt the LLM to produce both a confidence score $P^{u_i}_{v_k}$ and a rationale $R^{u_i}_{v_k}$ for each item v_k. Three human evaluators then assess the reasonableness of each rationale $R^{u_i}_{v_k}$ and assign an average quality score $Q^{u_i}_{v_k}$ ranging from 0 to 1. In this manner, we collect paired confidence scores and rationale quality scores for all test samples across three tasks, denoted as $C = \{P^{u_i}_{v_1}, P^{u_i}_{v_2}, \ldots, P^{u_i}_{v_n}\}$ and $Q = \{Q^{u_i}_{v_1}, Q^{u_i}_{v_2}, \ldots, Q^{u_i}_{v_n}\}$. Finally, we analyze the correlation coefficient between the confidence scores C and the quality scores of rationales Q. Figure 4 reveals moderate correlations between the confidence scores and reliability scores across the three tasks, with

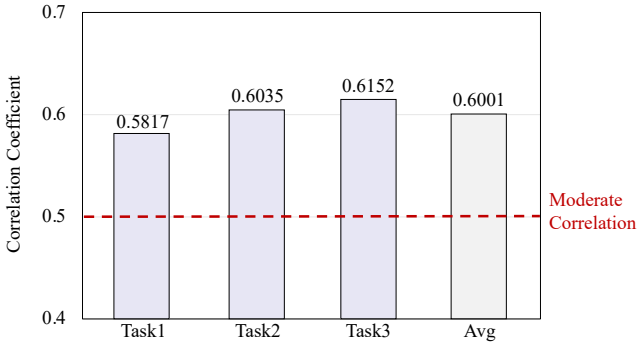

Figure 4: Correlation coefficient between confidence scores of CGI and knowledge transfer reliability scores of the LLM.

coefficients ranging from 0.5817 to 0.6152 (average: 0.6001). This indicates that by filtering out interactions with low confidence scores, CGI effectively mitigates unreliable knowledge transfer, leading to more accurate estimations of target-domain user preferences.

5.5 Hyper-parameter Analysis (RQ3)

In this section, we investigate the impact of key hyper-parameters on the performance of Lasso and analyze its characteristics.

Number of Simulated Interactions. We first examine how the performance of Lasso varies with different number of simulated interactions. Figure 5 illustrates the performance trends as the number of simulated interactions increases. Our analysis reveals that increasing the number of simulated interactions generally improves performance compared to the "BASE" model across all three tasks. Specifically, in Tasks 1 and 3, Lasso achieves peak performance as the number of simulated interactions increases from 0 to 40. Beyond this point, performance stabilizes and consistently outperforms the "BASE" variant. In Task 2, performance gradually peaks but declines with more interactions. We attribute this decline to the challenges in capturing cross-domain preferences between the book and music domains, as evidenced by the metrics in Table 2. This difficulty may lead to an overabundance of noisy simulated interactions, which can negatively impact downstream model performance.

Confidence Threshold in CGI. We also analyze the effect of varying confidence thresholds within the CGI mechanism on Lasso's performance. As shown in Figure 6, the performance of Lasso improves as the threshold increases from 0.5 to 0.7, which is attributed to the effective filtering of low-quality simulated interactions, reducing noise. However, when the threshold exceeds 0.7, performance begins to deteriorate. This is because nearly 80% of simulated interactions have confidence scores below 0.7, leading to the exclusion of most interactions and resulting in performance levels similar to the original cold-start scenario of the downstream model.

5.6 In-depth Analysis (RQ4)

The high quality of simulated interactions means that these interactions are both effective (improving the performance of the downstream recommendation) and generalizable (applicable across different models and scenarios). We first explore the effectiveness of

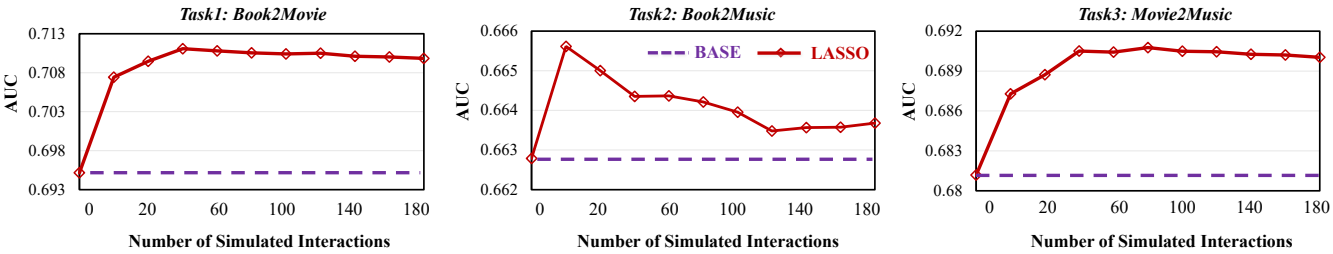

Figure 5: Performance with different number of simulated interactions. *BASE* represents the original recommendation model without simulated interactions as input.

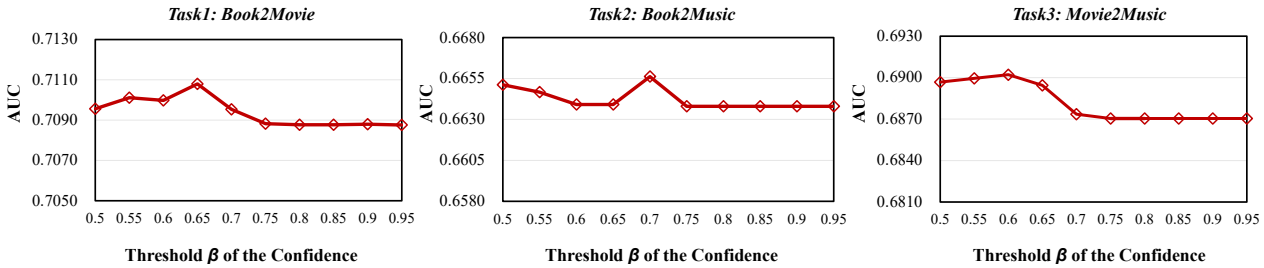

Figure 6: Performance of the Lasso with different confidence thresholds used in CGI module.

Table 4: Performance of the Lasso using different downstream recommendation models. For example, Lasso_{DIN} indicates Lasso with DIN model.

Model	*Task1*		*Task2*		*Task3*	
	AUC	NDCG	AUC	NDCG	AUC	NDCG
DIN	0.6952	0.8124	0.6628	0.7933	0.6812	0.8707
Lasso_{DIN}	**0.7108**	**0.8214**	**0.6656**	**0.7961**	**0.6902**	**0.8804**
DIEN	0.7161	0.8024	0.6489	0.5980	0.6781	0.8603
Lasso_{DIEN}	**0.7169**	**0.8125**	**0.6762**	**0.6204**	**0.6821**	**0.8605**
DeepFM	0.4952	0.4876	0.5137	0.5019	0.4933	0.4900
Lasso_{DeepFM}	**0.5299**	**0.5199**	**0.5181**	**0.5322**	**0.5079**	**0.4960**
BST	0.7194	0.8143	0.6642	0.7723	0.6788	0.8633
Lasso_{BST}	**0.7228**	**0.8205**	**0.6776**	**0.7812**	**0.6852**	**0.8635**

Table 5: Performance comparison of the LLM-based simulator against the baseline.

Model	*Task1*		*Task2*		*Task3*	
	AUC	NDCG	AUC	NDCG	AUC	NDCG
TMCDR	0.6841	0.7712	0.6631	0.6713	0.6668	0.6761
Lasso_s	**0.7473**	**0.8846**	**0.6831**	**0.7574**	**0.7164**	**0.8466**

Lasso across various traditional downstream models [6, 10, 35, 36], using the simulated interactions generated by our Lasso as inputs. As shown in Table 4, Lasso consistently improves the performance of various downstream models across all three evaluation tasks. Specifically, the integration of simulated interactions leads to an average increase of +0.9% in AUC for Lasso_{DIN}, +1.07% for Lasso_{DIEN} and +0.77% for Lasso_{BST}. Notably, DeepFM performs nearly at random without simulated interactions due to its poor performance in cold-start scenarios [14]. However, with the inclusion of these interactions as training data, DeepFM achieves a substantial improvement of 1.79% in AUC, demonstrating the effectiveness of simulated interactions generated by Lasso.

We further evaluate the quality of simulated interactions by directly assessing the performance of the LLM-based simulator. Specifically, we employ the LLM-based simulator, denoted as Lasso_s, to directly predict items that user may like in the target domain, rather than generating simulated interactions for downstream models. We compare this approach with the leading baseline, TMCDR, and present the results in Table 5. Our findings reveal that the LLM-based simulator outperforms TMCDR, demonstrating its effectiveness in capturing user preferences and mimicking user behaviors. This insight explains why Lasso effectively enhances downstream models: the simulated interactions accurately identify items that users genuinely value, providing more meaningful data. However, directly using the LLM as a online model may introduce latency issues [17, 28]. Therefore, we leverage the LLM as a simulator to enhance the performance of lightweight downstream models rather than deploying it as an online model.

6 Real-world Industrial Application

This section investigates the real-world applicability of Lasso within the context of the Kwai E-commerce platform [3], a prominent E-commerce platform in China. Our analysis focuses on transferring user preferences from the live-streaming domain (source domain) to the task of product recommendation (target domain). Specifically, we aim to leverage users' extensive live-streaming interactions, including clicks and viewing history, to enhance product recommendation accuracy, denoted as *Live Stream* → *Product*. Notably,

[3] https://www.kwaixiaodian.com/

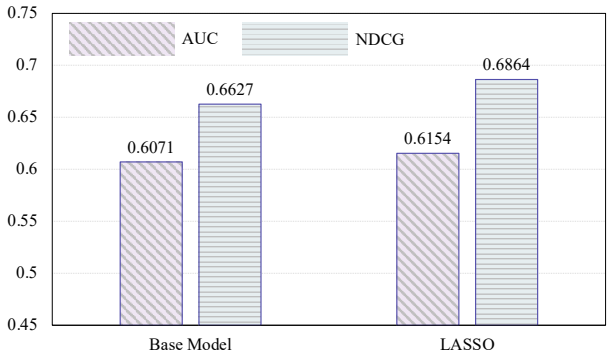

Figure 7: Performance of the Lasso and base model in the real-world industrial application.

this industrial application presents a unique challenge compared to established benchmarks like Amazon, as the LLM may not have encountered data from such a specific domain during its pre-training. This highlights the need for adaptability and generalization capabilities within Lasso.

Experimental Setup. We collect user interactions from past one month, and filter out users and items with fewer than 10 positive interactions. This results in a dataset comprising 500K overlapping users and 2 million entries. The training and test sets are divided using the same methodology as the Amazon review dataset. We utilize one of the online recommendation models deployed on the Kwai platform as the base model, which takes users' historical interactions as input.

Experimental Results. Figure 7 illustrates the performance of the base model and Lasso, where the Lasso denotes the base model enhanced by simulated interactions. We find that Lasso effectively enhances the performance of the base model. Specifically, Lasso increases the base model's AUC by 0.83% and NDCG by 2.37%. These results demonstrate the effectiveness of Lasso in a real-world industrial setting, highlighting its ability to improve recommendation system performance in cold-start scenarios by leveraging user behaviors from non-cold-start contexts.

7 Complexity Analysis

The computational complexity of applying Lasso in downstream models comprises three main components: personalized candidate pool construction complexity, confidence-guided inference complexity, and application complexity of the simulated interactions.

Construction Complexity. Using the parallel similarity retrieval framework, we can efficiently identify users with similar cross-domain preferences and their preferred target domain items, reducing the candidate set from billions to hundreds in $O(1)$ time.

Inference Complexity. Our confidence-guided inference mechanism filters low-confidence interactions, with complexity dominated by LLM inference (approximately 50-100 ms per user-item pair). In practice, we employ quantized models with parallel batch processing for offline execution. The resulting simulated interactions are stored in a database for efficient access.

Application Complexity. During downstream model deployment, simulated interactions are retrieved in constant time $O(1)$ through user-ID indexed database queries.

8 CONCLUSION

In this paper, we propose Lasso, a novel framework designed to enhance data efficiency by leveraging large language models (LLMs). First, we employ a cross-domain training paradigm to fine-tune the LLM as a user simulator, enabling it to adapt effectively to cross-domain recommendation scenarios. Then, a personalized candidate pool is constructed using cross-domain collaborative filtering, ensuring efficient simulation. The LLM-based user simulator utilizes the confidence-guided inference mechanism to infer the preferences of the cold-start user for each item in the candidate pool, using the real source domain interactions as inputs. Finally, the simulated interactions generated by Lasso are integrated into downstream recommendation models, significantly enhancing their performance in the target domain. Extensive experiments on both public and real-world datasets validate the effectiveness and compatibility of Lasso. This work represents an initial step toward improving the data efficiency of CDR using LLMs. Future research will focus on exploring additional characteristics of Lasso, including its scalability across different LLM backbones and the optimization of hyper-parameter selection.

References

[1] Qingyao Ai, Ting Bai, Zhao Cao, Yi Chang, Jiawei Chen, Zhumin Chen, Zhiyong Cheng, Shoubin Dong, Zhicheng Dou, Fuli Feng, et al. 2023. Information retrieval meets large language models: a strategic report from chinese ir community. *AI Open* 4 (2023), 80–90.

[2] Keqin Bao, Jizhi Zhang, Yang Zhang, Wenjie Wang, Fuli Feng, and Xiangnan He. 2023. Tallrec: An effective and efficient tuning framework to align large language model with recommendation. In *Proceedings of the 17th ACM Conference on Recommender Systems*. 1007–1014.

[3] Iván Cantador and Paolo Cremonesi. 2014. Tutorial on cross-domain recommender systems. In *Proceedings of the 8th ACM Conference on Recommender Systems*. 401–402.

[4] Jiangxia Cao, Shaoshuai Li, Bowen Yu, Xiaobo Guo, Tingwen Liu, and Bin Wang. 2023. Towards universal cross-domain recommendation. In *Proceedings of the Sixteenth ACM International Conference on Web Search and Data Mining*. 78–86.

[5] Jiangxia Cao, Xixun Lin, Xin Cong, Jing Ya, Tingwen Liu, and Bin Wang. 2022. Disencdr: Learning disentangled representations for cross-domain recommendation. In *Proceedings of the 45th International ACM SIGIR conference on research and development in information retrieval*. 267–277.

[6] Qiwei Chen, Huan Zhao, Wei Li, Pipei Huang, and Wenwu Ou. 2019. Behavior sequence transformer for e-commerce recommendation in alibaba. In *Proceedings of the 1st international workshop on deep learning practice for high-dimensional sparse data*. 1–4.

[7] Yanda Chen, Ruiqi Zhong, Sheng Zha, George Karypis, and He He. 2022. Meta-learning via Language Model In-context Tuning. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Smaranda Muresan, Preslav Nakov, and Aline Villavicencio (Eds.). Association for Computational Linguistics, Dublin, Ireland, 719–730. doi:10.18653/v1/2022.acl-long.53

[8] Chelsea Finn, Pieter Abbeel, and Sergey Levine. 2017. Model-agnostic meta-learning for fast adaptation of deep networks. In *International conference on machine learning*. PMLR, 1126–1135.

[9] Yunfan Gao, Tao Sheng, Youlin Xiang, Yun Xiong, Haofen Wang, and Jiawei Zhang. 2023. Chat-rec: Towards interactive and explainable llms-augmented recommender system. *arXiv preprint arXiv:2303.14524* (2023).

[10] Huifeng Guo, Ruiming Tang, Yunming Ye, Zhenguo Li, and Xiuqiang He. 2017. DeepFM: a factorization-machine based neural network for CTR prediction. *arXiv preprint arXiv:1703.04247* (2017).

[11] Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yongdong Zhang, and Meng Wang. 2020. Lightgcn: Simplifying and powering graph convolution network for recommendation. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*. 639–648.

[12] Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. 2017. Neural collaborative filtering. In *Proceedings of the 26th international conference on world wide web*. 173–182.

[13] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685* (2021).

[14] Feiran Huang, Zhenghang Yang, Junyi Jiang, Yuanchen Bei, Yijie Zhang, and Hao Chen. 2024. Large Language Model Interaction Simulator for Cold-Start Item Recommendation. *arXiv preprint arXiv:2402.09176* (2024).

[15] SeongKu Kang, Junyoung Hwang, Dongha Lee, and Hwanjo Yu. 2019. Semi-supervised learning for cross-domain recommendation to cold-start users. In *Proceedings of the 28th ACM international conference on information and knowledge management*. 1563–1572.

[16] Chenglin Li, Yuanzhen Xie, Chenyun Yu, Bo Hu, Zang Li, Guoqiang Shu, Xiaohu Qie, and Di Niu. 2023. One for all, all for one: Learning and transferring user embeddings for cross-domain recommendation. In *Proceedings of the Sixteenth ACM International Conference on Web Search and Data Mining*. 366–374.

[17] Xiangyang Li, Bo Chen, Lu Hou, and Ruiming Tang. 2023. CTRL: Connect Collaborative and Language Model for CTR Prediction. *arXiv preprint arXiv:2306.02841* (2023).

[18] Jianghao Lin, Xinyi Dai, Yunjia Xi, Weiwen Liu, Bo Chen, Hao Zhang, Yong Liu, Chuhan Wu, Xiangyang Li, Chenxu Zhu, et al. 2023. How can recommender systems benefit from large language models: A survey. *arXiv preprint arXiv:2306.05817* (2023).

[19] Weiming Liu, Chaochao Chen, Xinting Liao, Mengling Hu, Jiajie Su, Yanchao Tan, and Fan Wang. 2024. User Distribution Mapping Modelling with Collaborative Filtering for Cross Domain Recommendation. In *Proceedings of the ACM on Web Conference 2024*. 334–343.

[20] Kezhi Lu, Qian Zhang, Danny Hughes, Guangquan Zhang, and Jie Lu. 2024. AMT-CDR: A Deep Adversarial Multi-channel Transfer Network for Cross-domain Recommendation. *ACM Transactions on Intelligent Systems and Technology* (2024).

[21] Tong Man, Huawei Shen, Xiaolong Jin, and Xueqi Cheng. 2017. Cross-domain recommendation: An embedding and mapping approach. In *IJCAI*, Vol. 17. 2464–2470.

[22] Xuying Ning, Wujiang Xu, Xiaolei Liu, Mingming Ha, Qiongxu Ma, Youru Li, Linxun Chen, and Yongfeng Zhang. 2024. Information Maximization via Variational Autoencoders for Cross-Domain Recommendation. *arXiv preprint arXiv:2405.20710* (2024).

[23] Alessandro Petruzzelli, Cataldo Musto, Lucrezia Laraspata, Ivan Rinaldi, Marco de Gemmis, Pasquale Lops, and Giovanni Semeraro. 2024. Instructing and prompting large language models for explainable cross-domain recommendations. In *Proceedings of the 18th ACM Conference on Recommender Systems*. 298–308.

[24] Zuoli Tang, Zhaoxin Huan, Zihao Li, Xiaolu Zhang, Jun Hu, Chilin Fu, Jun Zhou, and Chenliang Li. 2023. One model for all: Large language models are domain-agnostic recommendation systems. *arXiv preprint arXiv:2310.14304* (2023).

[25] SM Tonmoy, SM Zaman, Vinija Jain, Anku Rani, Vipula Rawte, Aman Chadha, and Amitava Das. 2024. A comprehensive survey of hallucination mitigation techniques in large language models. *arXiv preprint arXiv:2401.01313* (2024).

[26] Ajay Krishna Vajjala, Dipak Falgun Meher, Shrunal Pothagoni, Ziwei Zhu, and David S Rosenblum. 2024. Vietoris-rips complex: A new direction for cross-domain cold-start recommendation. In *Proceedings of the 2024 SIAM International Conference on Data Mining (SDM)*. SIAM, 761–769.

[27] Shuohang Wang, Yang Liu, Yichong Xu, Chenguang Zhu, and Michael Zeng. 2021. Want to reduce labeling cost? GPT-3 can help. *arXiv preprint arXiv:2108.13487* (2021).

[28] Yunjia Xi, Weiwen Liu, Jianghao Lin, Xiaoling Cai, Hong Zhu, Jieming Zhu, Bo Chen, Ruiming Tang, Weinan Zhang, Rui Zhang, et al. 2023. Towards open-world recommendation with knowledge augmentation from large language models. *arXiv preprint arXiv:2306.10933* (2023).

[29] Wenhao Yang, Yingchun Jian, Yibo Wang, Shiyin Lu, Lei Shen, Bing Wang, Haihong Tang, and Lijun Zhang. 2024. Not All Embeddings are Created Equal: Towards Robust Cross-domain Recommendation via Contrastive Learning. In *Proceedings of the ACM on Web Conference 2024*. 3195–3206.

[30] Junjie Zhang, Ruobing Xie, Yupeng Hou, Wayne Xin Zhao, Leyu Lin, and Ji-Rong Wen. 2023. Recommendation as instruction following: A large language model empowered recommendation approach. *arXiv preprint arXiv:2305.07001* (2023).

[31] Yang Zhang, Fuli Feng, Jizhi Zhang, Keqin Bao, Qifan Wang, and Xiangnan He. 2023. Collm: Integrating collaborative embeddings into large language models for recommendation. *arXiv preprint arXiv:2310.19488* (2023).

[32] Cheng Zhao, Chenliang Li, Rong Xiao, Hongbo Deng, and Aixin Sun. 2020. CATN: Cross-domain recommendation for cold-start users via aspect transfer network. In *Proceedings of the 43rd international ACM SIGIR conference on research and development in information retrieval*. 229–238.

[33] Chuang Zhao, Hongke Zhao, Xiaomeng Li, Ming He, Jiahui Wang, and Jianping Fan. 2023. Cross-domain recommendation via progressive structural alignment. *IEEE Transactions on Knowledge and Data Engineering* (2023).

[34] Zihuai Zhao, Wenqi Fan, Jiatong Li, Yunqing Liu, Xiaowei Mei, Yiqi Wang, Zhen Wen, Fei Wang, Xiangyu Zhao, Jiliang Tang, et al. 2023. Recommender systems in the era of large language models (llms). *arXiv preprint arXiv:2307.02046* (2023).

[35] Guorui Zhou, Na Mou, Ying Fan, Qi Pi, Weijie Bian, Chang Zhou, Xiaoqiang Zhu, and Kun Gai. 2019. Deep interest evolution network for click-through rate prediction. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 33. 5941–5948.

[36] Guorui Zhou, Xiaoqiang Zhu, Chenru Song, Ying Fan, Han Zhu, Xiao Ma, Yanghui Yan, Junqi Jin, Han Li, and Kun Gai. 2018. Deep interest network for click-through rate prediction. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 1059–1068.

[37] Feng Zhu, Yan Wang, Chaochao Chen, Jun Zhou, Longfei Li, and Guanfeng Liu. 2021. Cross-domain recommendation: challenges, progress, and prospects. *arXiv preprint arXiv:2103.01696* (2021).

[38] Feng Zhu, Yan Wang, Jun Zhou, Chaochao Chen, Longfei Li, and Guanfeng Liu. 2021. A unified framework for cross-domain and cross-system recommendations. *IEEE Transactions on Knowledge and Data Engineering* (2021).

[39] Yongchun Zhu, Kaikai Ge, Fuzhen Zhuang, Ruobing Xie, Dongbo Xi, Xu Zhang, Leyu Lin, and Qing He. 2021. Transfer-meta framework for cross-domain recommendation to cold-start users. In *Proceedings of the 44th international ACM SIGIR conference on research and development in information retrieval*. 1813–1817.

[40] Yongchun Zhu, Zhenwei Tang, Yudan Liu, Fuzhen Zhuang, Ruobing Xie, Xu Zhang, Leyu Lin, and Qing He. 2022. Personalized transfer of user preferences for cross-domain recommendation. In *Proceedings of the fifteenth ACM international conference on web search and data mining*. 1507–1515.